

Why do we need transformers?

Understanding natural language often goes beyond simple word meanings. **Context is crucial** in determining how a word contributes to a sentence's meaning.

Example – Do not take the medicine, as it has expired.

What is the relationship between “medicine” and each of the words in the sentence, do, not, take, the, medicine (yes medicine itself) and so on. How do we realize that medicine and expired have a connection?

RNN and LSTM networks are widely used in sequential tasks such as next word prediction, machine translation, text generation, and more. However, one of the major challenges with the **recurrent model** is **capturing the long-term dependency** in a sequence.

LSTMs process information sequentially, passing data through a chain of cells. This can make it **challenging to capture relationships between elements far apart in a sequence**.

LSTMs process information step-by-step – **sequential nature**, limiting their parallelism. This can be slow for long sequences.

LSTMs can suffer from vanishing or exploding gradients during training, making it difficult to learn long-term dependencies. Transformers address this by ~~not relying solely on sequential information flow~~.

RNNs are inherently **sequential** – you can't process the next step until the previous one is done. This can make them **slow, especially for long sequences**

To overcome this limitation of RNNs, a new architecture called Transformer was introduced in the paper “Attention Is All You Need.”

The transformer model is based primarily on a special type of attention mechanism, the self-attention mechanism, which allows it to process information without relying solely on the sequential order of words in the input. This is a significant departure from traditional recurrent neural networks (RNNs) that depend heavily on recurrent connections.

Self-attention allows the model to consider/attend to any part of the input sequence. It **calculates a score for each position indicating its relevance to the current element being processed**. Higher scores indicate stronger attention. This allows the model to directly **calculate dependencies** between any **two positions** in a **sequence, regardless of distance**.

Self-attention, act as a mixer as it **mixes information between different positions** in the input sequence. This lets the model learn nuanced relationships and focus on the most relevant parts of the input for a given task.

This helps it understand long-range context, especially important for language understanding. This makes them much **more effective at understanding long-range dependencies**, crucial for tasks like language understanding, machine translation and document summarization.

Consider the sentence: "The movie was visually stunning but the acting felt a bit wooden."

An RNN would process the sentence word by word. It might miss the connection between "visually stunning" at the beginning and "wooden" at the end when it reaches "felt." Using self-attention, the model can calculate the relevance of "visually stunning" when processing "wooden" and vice versa. This allows it to understand the contrasting sentiment between these two phrases, even though they are separated by other words.

Self-attention calculations can be performed in parallel for each position in the sequence. This makes it **computationally efficient** and allows it to handle larger datasets and longer sequences effectively.

Architectures such as BERT, GPT-3, T5, and more are based on Transformer.

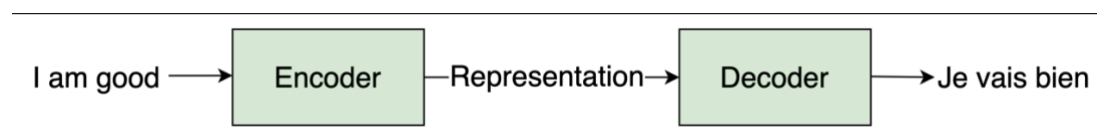
How does the transformer work

Let's understand how the transformer works with a language translation task. The transformer consists of an **encoder-decoder architecture**. We feed the input sentence (source sentence) to the encoder. The encoder

- **learns** the **representation** of the **input** sentence and
- **sends** the **representation** to the **decoder**.

The decoder receives the representation learned by the encoder as an input and generates the output sentence (target sentence).

Suppose we need to convert a sentence from English to French. We feed the English sentence as input to the encoder. The encoder learns the representation of the given English sentence and feeds the representation to the decoder. The decoder takes the encoder's representation as input and generates the French sentence as output:



Why Encoder-Decoder separately –

Encoder: Understanding the Input: The encoder's primary job is to **analyze** the **input sequence** and **create** a rich, **contextual representation** of it. This means it learns to **identify** important **relationships, patterns, or meaning** within the input data.

Decoder: Generating the Output: The decoder focuses on taking the encoder's representation as its starting point and generating a new output sequence (e.g., a translation, a summary, or an answer). This process is often **autoregressive**, where the **previously generated element** is used as **input** for the next step.

Encoders and decoders enable models to handle tasks where the **input** sequence and **output sequence** might be of **different lengths**. This is critical in tasks like **machine translation** where the target language sentence might be shorter or longer than the input sentence.

This architecture allows you to potentially switch out the decoder for different tasks while keeping the encoder trained on understanding the input format. For example, the same encoder from a translation model might be reused with a different decoder for tasks like summarization or question-answering.

The **attention mechanism**, which is central to Transformers, lets the **decoder** intelligently **focus** on **different parts** of the **encoder's output** during different stages of output **generation**. This lets it **selectively draw information** most **relevant to the current word** or phrase being generated.

How does the encoder and decoder in the transformer convert the English sentence (source sentence) to the French sentence (target sentence)?

Encoder: Reads the French sentence, creating a representation that captures the meaning and nuances of the original language.

Decoder: Takes the encoder's representation and generates the English translation word-by-word. Attention helps the decoder determine which parts of the French sentence are most important when generating each English word.

What's going on inside the encoder and decoder?

The transformer consists of a stack of N number of encoders. The output of one encoder is sent as input to the encoder above it.

Why?

Capturing Complex Relationships:

1. Hierarchical Features: **Stacking** encoders allows the model to learn **increasingly complex representations** of the input data.

Early encoders might focus on simple relationships, while **later encoders** build upon these to capture **higher-level, more abstract patterns**. This is similar to how deeper convolutional neural networks extract increasingly complex features from images.

2. Long-Range Dependencies: In tasks like natural language processing, **relationships** between words or elements can **span long distances within a sequence**. A single encoder may struggle with these long-range dependencies.

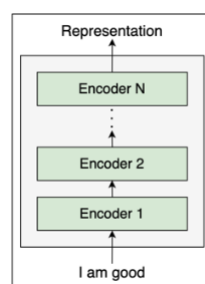
Multiple encoders **progressively distill information**, allowing later encoders to attend to distant but relevant parts of the input.

Improved Generalization:

Regularization: Multiple encoders can act as a form of regularization, helping to prevent **overfitting** to the training data. Each encoder has the opportunity to **learn somewhat different representations**, making the model **less reliant on any single layer's output**.

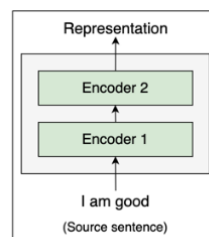
As shown in the following figure, we have a stack of N number of encoders. Each encoder sends its output to the encoder above it. The final encoder returns the representation of the given source sentence as output.

We feed the source sentence as input to the encoder and get the representation of the source sentence as output:



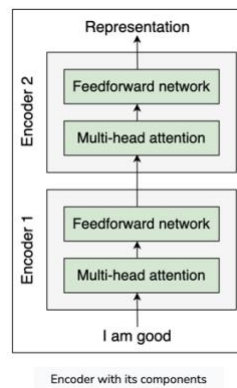
A stack of N number of encoders

Note that in the transformer paper "[Attention Is All You Need](#)," the authors have used $N=6$, meaning that they stacked up six encoders, one above the other. However, we can try out different values of N . For simplicity and better understanding, let's keep $N=2$:



A stack of encoders

how exactly does the encoder work?



Each encoder block consists of two sublayers:

- Multi-head attention
- Feedforward network

To understand how multi-head attention works, we first need to understand the self-attention mechanism.

Let's understand the self-attention mechanism with an example. Consider the following sentence:
The dog ate the food because it was hungry

In the preceding sentence, the pronoun 'it' could mean either 'dog' or 'food'. By reading the sentence, we can easily understand that the pronoun 'it' implies the 'dog' and not 'food'. But how does our model understand that in the given sentence, the pronoun 'it' implies the 'dog' and not 'food'?

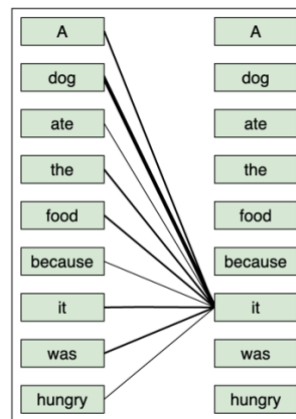
Here is where the self-attention mechanism helps us.

In the given sentence, 'A dog ate the food because it was hungry', first our model computes the representation of the word 'A', next it computes the representation of the word 'dog', then it computes the representation of the word 'ate', and so on. While computing the representation of each word, it relates each word to all other words in the sentence to understand more about the word.

For instance, while computing the representation of the word 'it', our model relates the word 'it' to all the words in the sentence.

By relating the word 'it' to all the words in the sentence, our model can understand that the word 'it' is related to the word 'dog' and not 'food'.

As we can observe, the line connecting the word 'it' to 'dog' is thicker compared to the other lines, which indicates that the word 'it' is related to the word 'dog' and not 'food' in the given sentence:



Self-attention example

Okay, but how exactly does this work?

What is embedding?

Suppose our input sentence (source sentence) is '*I am good*'. First, we get the embeddings for each word in our sentence. Note that the **embeddings** are just the **vector representation of the word** and the **values of the embeddings will be learned during training**.

Let x_1 be the embedding of the word '*I*', x_2 be the embedding of the word '*am*', and x_3 be the embedding of the word '*good*'. Consider the following:

- The embedding of the word '*I*' is $x_1 = [1.76, 2.22, \dots, 6.66]$.
- The embedding of the word '*am*' is $x_2 = [7.77, 0.631, \dots, 5.35]$.
- The embedding of the word '*good*' is $x_3 = [11.44, 10.10, \dots, 3.33]$.

Then, we can represent our input sentence '*I am good*' using the input matrix X (embedding matrix or input embedding), as shown here:

I	1.76	2.22	...	6.66	x_1
am	7.77	0.631	...	5.35	x_2
good	11.44	10.10	...	3.33	x_3
					3x512

X
input matrix
(embedding matrix)

Input matrix

From the preceding input matrix X , we can understand that the first row of the matrix implies the embedding of the word '*I*', the second row implies the embedding of the word '*am*', and the third row implies the embedding of the word '*good*'.

Dimensions of the embedding matrix

The dimension of the input matrix X will be:

[sentence length $\times$ embedding dimension]

The number of words in our sentence (sentence length) is 3. Let the embedding dimension be 512; then, our input matrix (input embedding) dimension will be:

[3×512]

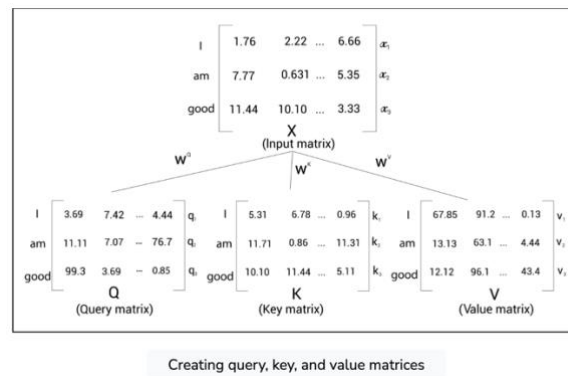
Now, from the input matrix X , we create three new matrices: a query matrix Q , key matrix K , and a value matrix V . Wait.

What are these three new matrices? And why do we need them? They are used in the self-attention mechanism.

How can we create the query, key, and value matrices? To create these, we introduce three new weight matrices called WQ , WK , WV . We create the query, Q , key, K , and value, V , matrices by multiplying the input matrix X by WQ , WK , and WV respectively.

Note: The weight matrices WQ , WK , and WV are **randomly initialized**, and their optimal values will be **learned during training**. As we learn the optimal weights, we will obtain more accurate query, key, and value matrices.

As shown in the following figure, multiplying the input matrix X , by the weight matrices WQ , WK , and WV , we obtain the query, key, and value matrices:



From the preceding figure, we can understand the following:

- The first row in the query, key, and value matrices— q_1 , k_1 , and v_1 —implies the query, key, and value vectors of the word 'I'.
- The second row in the query, key, and value matrices— q_2 , k_2 , and v_2 —implies the query, key, and value vectors of the word 'am'.
- The third row in the query, key, and value matrices— q_3 , k_3 , and v_3 —implies the query, key, and value vectors of the word 'good'.

Dimensions of the query, key, and value matrices

Note that the dimensionality of the query, key, and value vectors is 64. Thus, the dimension of our query, key, and value matrices is:

[sentence length×64]

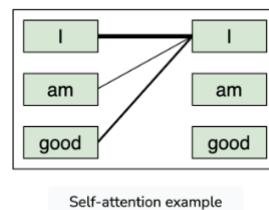
Since we have three words in the sentence, the dimensions of the query, key, and value matrices are:

[3×64]

How the self-attention mechanism works

We learned how to compute query, Q , key, K , and value, V , matrices and we also learned that they are obtained from the input matrix, X . Now, let's see how the query, key, and value matrices are used in the self-attention mechanism.

We learned that in order to compute a representation of a word, the self-attention mechanism relates the word to all the words in the given sentence. Consider the sentence '*I am good*'. To compute the representation of the word '*I*', we relate the word '*I*' to all the words in the sentence, as shown in the following figure:



But why do we need to do this? Understanding how a word is related to all the words in the sentence helps us to learn better representation. Now, let's learn how the self-attention mechanism relates a word to all the words in the sentence using the query, key, and value matrices. The self-attention mechanism includes four steps. Let's take a look at them one by one.

Step 1: Calculating the similarity score

The first step in the self-attention mechanism is to compute the dot product between the query matrix, Q , and the key matrix, K^T :

$$\begin{array}{c}
 \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \begin{array}{c} q_1 \\ q_2 \\ q_3 \end{array} \\
 Q
 \end{array} \cdot \begin{array}{c} \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \begin{array}{c} k_1 \\ k_2 \\ k_3 \end{array} \\
 K^T
 \end{array}$$

Query and key matrices

The following shows the result of the dot product between the query matrix, Q , and the key matrix, K^T :

$$\begin{array}{c} \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \begin{array}{c} q_1 \\ q_2 \\ q_3 \end{array} \\ Q \end{array} \cdot \begin{array}{c} \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \begin{array}{c} k_1 \\ k_2 \\ k_3 \end{array} \\ K^T \end{array} = \begin{array}{c} \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & q_1 \cdot k_3 \\ q_2 \cdot k_1 & q_2 \cdot k_2 & q_2 \cdot k_3 \\ q_3 \cdot k_1 & q_3 \cdot k_2 & q_3 \cdot k_3 \end{bmatrix} \\ QK^T \end{array} = \begin{array}{c} \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix} \end{array}$$

Computing the dot product between the query and key matrices

But what is the use of computing the dot product between the query and key matrices? What exactly does $Q.KT$ signify? Let's understand this by looking at the result of $Q.KT$ in detail.

The similarity score for 'I'

Let's look into the first row of the $Q.KT$ matrix as shown in the following figure. We can observe that we are computing the dot product between the query vector, $q1(I)$, and all the key vectors— $k1(I)$, $k2(am)$, and $k3(good)$. **Computing the dot product between two vectors tells us how similar they are.**

Therefore, computing the dot product between the query vector ($q1$) and the key vectors ($k1, k2, k3$) tells us how similar the query vector $q1(I)$ is to all the key vectors— $k1(I)$, $k2(am)$, and $k3(good)$. By looking at the first row of the $Q.KT$ matrix, we can understand that the word 'I' is more related to itself than the words 'am' and 'good' since the dot product value is higher for $q1.k1$ compared to $q1.k2$ and $q1.k3$:

		I	am	good
$QK^T =$	I	110	90	80
		$q_1.k_1$	$q_1.k_2$	$q_1.k_3$
	am	70	99	70
	good	90	70	100

Computing the dot product between the query vector ($q1$) and the key vectors ($k1$, $k2$, and $k3$)

Computing the dot product between the query vector ($q1$) and the key vectors ($k1$, $k2$, and $k3$)

Note: The values used in the input matrices are arbitrary and used here just to give us a better understanding.

The similarity score for 'am'

Now, let's look into the second row of the $Q.KT$ matrix. As shown in the following figure, we can observe that we are computing the dot product between the query vector, $q2(am)$, and all the key vectors— $k1(I)$, $k2(am)$, and $k3(good)$. This tells us how similar the query vector $q2(am)$ is to the key vectors— $k1(I)$, $k2(am)$, and $k3(good)$.

By looking at the second row of the $Q.KT$ matrix, we can understand that the word 'am' is more related to itself than the words 'I' and 'good' since the dot product value is higher for $q2.k2$ compared to $q2.k1$ and $q2.k3$:

		I	am	good
$QK^T =$	I	110	90	80
	am	70	99	70
		$q_2.k_1$	$q_2.k_2$	$q_2.k_3$
	good	90	70	100

Computing dot product between the query vector ($q2$) and the key vectors ($k1$, $k2$, and $k3$)

The similarity score for 'good'

Similarly, let's look into the third row of the $Q.KT$ matrix. As shown in the following figure, we can observe that we are computing the dot product between the query vector $q3(good)$ and all the key vectors— $k1$

(I), $k_2(\text{am})$, and $k_3(\text{good})$. This tells us how similar the query vector $q_3(\text{good})$ is to all the key vectors— $k_1(\text{I})$, $k_2(\text{am})$, and $k_3(\text{good})$.

By looking at the third row of the $Q.KT$ matrix, we can understand that the word 'good' is more related to itself than the words 'I' and 'am' in the sentence since the dot product value is higher for $q_3.k_3$ compared to $q_3.k_1$ and $q_3.k_2$:

		I	am	good
	I	110	90	80
	am	70	99	70
	good	90	70	100
		$q_1.k_1$	$q_1.k_2$	$q_1.k_3$

Computing the dot product between the query vector (q_3) and the key vectors (k_1 , k_2 , and k_3)

Thus, we can say that computing the dot product between the query matrix Q , and the key matrix KT , essentially gives us the similarity score, which helps us to understand how similar each word in the sentence is to all other words.

Step 2: Obtaining the stable gradients

The next step in the self-attention mechanism is to divide the $Q.KT$ matrix by the square root of the dimension of the key vector. But why do we have to do that? This is useful in obtaining stable gradients.

In the context of machine learning and optimization “obtaining stable gradients” refers to the process of ensuring that the gradients (partial derivatives of loss function with respect to the. Model parameters) do not become too large or too small during training. This is crucial for effective and stable learning, as it prevents issues like gradient explosion or vanishing gradients which can derail the training process.

Gradients represent the direction and magnitude of the change needed in the model's parameters to minimize the loss function. However, during the training process, gradients can sometimes grow very large or shrink to near zero leading to two main issues –

1. Gradient Explosion – this occurs when the gradients become excessively large. It can cause the model parameters to oscillate wildly or diverge, making the learning process unstable and preventing the model from converging to a solution.
2. Vanishing Gradients – this happens when the gradients become too small, effectively halting the learning process because the model parameters hardly update. It is particularly problematic in deep networks where the gradient signal can diminish as it propagates back through the layers.

In the context of the lesson, which discusses the self-attention mechanism part of the Transformer architecture, the division of the dot product of the query and key matrices by the square root of the dimension of the key vector is a practical solution to mitigate the risk of unstable gradients. This operation is known as “scaling” and it has a specific purpose – Scaling the dot product operation in

the self-attention can lead to very large values, especially when the dimensionality of the key vectors is high. These large values, in turn, can lead to large gradients during backpropagation. By dividing the dot product by square root of d_k , the resulting values are scaled down, leading to a more moderate and controlled gradient scale. This helps in maintaining the stability of the gradient descent optimization process, ensuring that the updates to the model parameters are neither too large (which could overshoot the optimal point) nor too small (which could stall the learning process).

This technique is critical component in the design of transformer models, enabling them to learn effectively from data and achieve remarkable performance across a wide range of tasks in natural language processing and beyond.

Let d_k be the dimension of the key vector. Then, we divide $Q.K^T$ by d_k . The dimension of the key vector is 64. So, taking the square root of it, we will obtain 8. Therefore, we divide $Q.K^T$ by 8 as shown in the following figure:

$$\frac{QK^T}{\sqrt{d_k}} = \frac{QK^T}{8} = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 13.75 & 11.25 & 10 \\ 8.75 & 12.375 & 8.75 \\ 11.25 & 8.75 & 12.5 \end{bmatrix} \end{matrix}$$

Obtaining the stable gradients

Step 3: Normalizing the gradients

By looking at the preceding similarity scores, we can understand that they are in the unnormalized form. So, we normalize them using the softmax function. Applying the softmax function helps in bringing the score to the range of 0 to 1. The sum of the scores equals 1, as shown in the following figure:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \end{matrix}$$

Applying the softmax function

We can call the preceding matrix a score matrix. With the help of these scores, we can understand how each word in the sentence is related to all the words in the sentence. For instance, look at the first row in the preceding score matrix. It tells us that the word 'I' is related to itself by 90%, to the word 'am' by 7%, and to the word 'good' by 3%.

Step 4: Computing the attention matrix

Okay, what's next? We computed the dot product between the query and key matrices, obtained the scores, and then normalized the scores with the softmax function. Now, the final step in the self-attention mechanism is to compute the attention matrix, Z .

Okay, what's next? We computed the dot product between the query and key matrices, obtained the scores, and then normalized the scores with the softmax function. Now, the final step in the self-attention mechanism is to compute the attention matrix, Z .

The attention matrix contains the attention values for each word in the sentence. We can compute the attention matrix, Z , by multiplying the score matrix, $\text{softmax}(dkQK^T)$, by the value matrix, V , as shown in the following figure:

$$Z = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \end{matrix} \quad \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

$$V = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 67.85 & 91.2 & \dots & 0.13 \\ 13.13 & 63.1 & \dots & 4.44 \\ 12.12 & 96.1 & \dots & 43.4 \end{bmatrix} \end{matrix} \quad \begin{matrix} v_1 \\ v_2 \\ v_3 \end{matrix}$$

Computing the attention matrix

Say we have the following

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \begin{matrix} I \\ am \\ good \end{matrix}$$

Result of the attention matrix

Attention matrix for 'I'

The attention matrix, Z , is computed by taking the sum of the value vectors weighted by the scores. Let's understand this by looking at it row by row. First, let's see how for the first row, z_1 , the self-attention of the word 'I' is computed:

$$z_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(I) \qquad \qquad v_2(am) \qquad \qquad v_3(good)$

Self-attention of the word I

From the preceding figure, we can understand that z_1 , the self-attention of the word 'I' is computed as the sum of the value vectors weighted by the scores. Therefore, the value of z_1 will contain 90% of the values from the value vector $v_1(I)$, 7% of the values from the value vector $v_2(am)$, and 3% of values from the value vector $v_3(good)$.

But how is this useful? To answer this question, let's take a little detour to the example sentence we saw earlier, 'A dog ate the food because it was hungry'. Here, the word 'it' indicates 'dog'. To compute the self-attention of the word 'it', we follow the same preceding steps. Suppose we have the following:

$$Z_{it} = 0.0 \begin{bmatrix} 71.1 & 6.1 & \dots \end{bmatrix} + 1.0 \begin{bmatrix} 31.1 & 11.1 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.9 & 11.44 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.8 & 12.44 & \dots \end{bmatrix}$$

$V_1(A)$ $V_2(\text{dog})$ $V_3(\text{food})$ $V_5(\text{hungry})$

Self-attention of the word it

Self-attention of the word it

From the preceding figure, we can understand that the self-attention value of the word 'it' contains 100% of the values from the value vector $v_2(\text{dog})$. This helps the model to understand that the word 'it' actually refers to 'dog' and not 'food'. Therefore, by using a self-attention mechanism, we can understand how a word is related to all other words in the sentence.

Attention matrix for 'am'

Now, coming back to our example, z_2 , the self-attention of the word 'am' is computed as the sum of the value vectors weighted by the scores, as shown in the following figure:

$$Z_2 = 0.025 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.95 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.025 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(I)$ $v_2(\text{am})$ $v_3(\text{good})$

Self-attention of the word am

As we can observe from the preceding figure, the value of z_2 will contain 2.5% of the values from the value vector $v_1(I)$, 95% of the values from the value vector $v_2(\text{am})$, and 2.5% of the values from the value vector $v_3(\text{good})$.

Attention matrix for 'good'

Similarly, z_3 , the self-attention of the word good is computed as the sum of the value vectors weighted by the scores, as shown in the following figure:

$$Z_3 = 0.21 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.76 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(I)$ $v_2(\text{am})$ $v_3(\text{good})$

Self-attention of the word good

This implies that the value of z_3 will contain 21% of the values from the value vector $v_1(I)$, 3% of the values from the value vector $v_2(\text{am})$, and 76% of values from the value vector $v_3(\text{good})$.

Therefore, the attention matrix, Z , consists of self-attention values of all the words in the sentence and it is computed as follows:

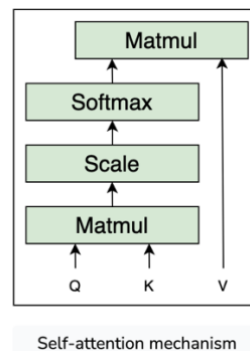
$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Summary

To get a better understanding of the self-attention mechanism, the steps involved are summarized as follows:

1. We compute the dot product between the query matrix and the key matrix, QK^T , and get the similarity scores.
2. We divide QK^T by the square root of the dimension of the key vector dk .
3. We apply the softmax function to normalize the scores and obtain the score matrix $\text{softmax}(QK^T/dk)$.
4. We compute the attention matrix, Z , by multiplying the score matrix by the value matrix V .

The self-attention mechanism is graphically shown as follows:



The self-attention mechanism is also called **scaled dot product attention**, since here we are computing the dot product (between the query and key vectors) and scaling the values (with dk).

Instead of computing a single attention matrix, Z , we can compute multiple attention matrices. But what is the use of computing multiple attention matrices?

Let's understand this with an example. Consider the phrase:

All is well

Say we need to compute the self-attention of the word 'well'. After computing the similarity score, suppose we have the following:

$$Z_{\text{well}} = 0.6 \begin{bmatrix} 3.1 & 6.8 & \dots \end{bmatrix} + 0.0 \begin{bmatrix} 0.1 & 0.6 & \dots \end{bmatrix} + 0.4 \begin{bmatrix} 6.4 & 8.3 & \dots \end{bmatrix}$$

$v_1(\text{All})$
 $v_2(\text{ is })$
 $v_5(\text{well})$

Self-attention of the word well

As we can observe from the preceding figure, the self-attention value of the word 'well' is the sum of the value vectors weighted by the scores. If you look at the preceding figure closely, the attention value of the actual word 'well' is dominated by the other word 'All'. That is, since we are multiplying the value vector of the word 'All' by 0.6 and the value vector of the actual word 'well' by only 0.4, it implies that Z_{well} will contain 60% of the values from the value vector of the word 'All' and only 40% of the values from the value vector of the actual word 'well'. Thus, here the attention value of the actual word 'well' is dominated by the other word 'All'.

This will be useful only in circumstances where the meaning of the actual word is ambiguous. That is, consider the following sentence:

A dog ate the food because it was hungry

Say we are computing the self-attention for the word 'it'. After computing the similarity score, suppose we have the following:

$$Z_{it} = 0.0 \begin{bmatrix} 71.1 & 6.1 & \dots \end{bmatrix} + 1.0 \begin{bmatrix} 31.1 & 11.1 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.9 & 11.44 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.8 & 12.44 & \dots \end{bmatrix}$$

V_1 (A) V_2 (dog) V_3 (food) V_9 (hungry)

Self-attention of the word it

As we can observe from the preceding equation, the attention value of the word 'it' is just the value vector of the word 'dog'. Here, the attention value of the actual word 'it' is dominated by the word 'dog'. But this is fine here since the meaning of the word 'it' is ambiguous, as it may refer to either 'dog' or 'food'.

Thus, if the value vector of other words dominates the actual word in cases as shown in the preceding example, **where the actual word is ambiguous, then this dominance is useful; otherwise, it will cause an issue in understanding the right meaning of the word.**

So, in order to make sure that our results are accurate, instead of computing a single attention matrix, we will compute multiple attention matrices and then concatenate their results. The idea behind using multi-head attention is that instead of using a single attention head, if we use multiple attention heads, then our attention matrix will be more accurate. Let's explore this in more detail.

Computing attention matrix Z1

We learned that to compute the attention matrix, we create three new matrices, called query, key, and value matrices. To create the query, Q_1 , key, K_1 , and value, V_1 , matrices, we introduce three new weight matrices called W_1Q , W_1K , and W_1V . We create the query, key, and value matrices by multiplying the input matrix, X , by W_1Q , W_1K , and W_1V respectively.

Now, the attention matrix Z_1 can be computed as follows:

$$Z = \text{softmax} \left(\frac{Q_1 K_1^T}{\sqrt{d_k}} \right) V_1$$

Computing attention matrix Z2

Now, let's compute the second attention matrix, Z_2 .

To compute the attention matrix Z_2 , we create another set of query, Q_2 , key, K_2 , and value, V_2 , matrices. We introduce three new weight matrices called W_2Q , W_2K , and W_2V , and we create the query, key, and value matrices by multiplying the input matrix X by W_2Q , W_2K , and W_2V , respectively.

The attention matrix Z_2 can be computed as follows:

$$Z = \text{softmax} \left(\frac{Q_2 K_2^T}{\sqrt{d_k}} \right) V_2$$

Concatenating multiple attention matrices

Similarly, we can compute h number of attention matrices. Suppose we have eight attention matrices, Z_1 to Z_8 ; then, we can just concatenate all the attention heads (attention matrices) and multiply the result by a new weight matrix W_O , and create the final attention matrix as shown:

Multi-head attention = Concatenate ($Z_1, Z_2, \dots, Z_i, \dots, Z_8$) W_O

Why multi-head?

The Problem with Single Attention

While the standard attention mechanism effectively focuses on relevant parts of an input sequence, it has limitations:

One-Dimensional Focus: A single attention head might attend to only one specific aspect of a word or element within the sequence. For example, in a sentence, it might focus on just the subject-verb relationship or just the adjective-noun relationship.

Multi-head attention tackles this by introducing multiple attention heads working in parallel. Here's how it works:

- **Splitting Inputs:** The input (query, key, and value vectors) is split into multiple chunks.
- **Multiple Attention Heads:** Each chunk is fed into separate attention heads. This allows these heads to focus on different aspects of the relationships within the sequence independently. For example, one head might focus on subject-verb relations, another on adjective-noun pairings, and so on.
- **Concatenation and Transformation:** The outputs from each attention head are concatenated (combined) and then transformed using a linear layer. This produces the final output of the multi-head attention layer.

Benefits of Multi-Head Attention:

- **Capturing Diverse Relationships:** Multiple attention heads can learn to attend to different aspects of the input sequence, leading to a richer and more nuanced understanding of its internal relationships.
- **Improved Representation:** The final output of the multi-head attention layer produces a more robust representation of the input sequence, incorporating information from various attention heads.
- **Parallelization:** The computation for each attention head can happen in parallel, improving computational efficiency.

Analogy: Imagine you're trying to understand a complex concept. Having multiple experts, each specializing in a different area, explain this concept would give you a much more comprehensive understanding than relying on a single expert. Multi-head attention works in a similar way for the model.

Multi-head attention facilitates learning diverse relationships within a sequence by using multiple independent "heads" to focus on different aspects of the data. Here's a breakdown with an example to illustrate:

Conceptual Breakdown:

Imagine you're analyzing a sentence to understand its meaning. A single-headed attention mechanism would be like having one analyst focusing on a specific aspect, like grammar or subject-verb agreement. While this provides some information, a more comprehensive understanding requires considering multiple aspects.

Multi-head attention tackles this by introducing multiple analysts (heads) working in parallel. Each head analyzes the sentence through a different lens, capturing various relationships between words.

Example: Analyzing a Sentence

Consider the sentence: "The quick brown fox jumps over the lazy dog."

Single-Head Attention: Imagine a single analyst focusing on subject-verb agreement. They would pay close attention to the relationship between "fox" and "jumps" and likely assign high attention weights to these words. However, they might miss other essential relationships, like the adjective "quick" describing the "fox" or the prepositional phrase "over the lazy dog."

Multi-Head Attention: With multiple heads, each can analyze a different aspect:

- Head 1: Focuses on subject-verb agreement, similar to the single analyst, highlighting the "fox-jumps" connection.
- Head 2: Analyzes adjective-noun relationships, attending to "quick" and "fox."
- Head 3: Pays attention to prepositions and their objects, capturing the relationship between "over" and "the lazy dog."

Diverse Attention Weights:

Each head independently calculates attention weights for each word in the sentence. These weights represent how relevant a particular word is to the current word being analyzed, considering the head's specific focus. The final output combines the information from all heads, resulting in a richer understanding of the relationships within the sentence.

Benefits of Diverse Relationships:

- **Improved Representation:** By capturing various relationships, the model builds a more comprehensive representation of the sequence.
- **Flexibility:** The model isn't limited to pre-defined relationships. It can learn to identify and attend to different aspects based on the data it encounters during training.

Additional Points:

- The number of heads (n_heads) is a hyperparameter that can be tuned for specific tasks.
- Each head has its own set of weight matrices, allowing it to learn independent attention patterns.

In essence, multi-head attention acts like a team of specialists working together to gain a deeper understanding of the data by considering various perspectives. This capability is crucial for tasks like machine translation, where understanding different grammatical structures and word relationships is essential for accurate translation.

While there aren't strict "thumb rules" on the number of heads in multi-head attention, here are some guidelines and factors to consider when making this choice:

Common Practices:

- **Starting Point** A good starting point is often to set the number of heads so the key/query dimension per head (d_k) is 64. For example, if the original dimensionality (d_{model}) is 512, using 8 heads would be a reasonable place to begin experimenting.
- **Range:** Commonly, the number of heads ranges between 4 and 16.
- **Powers of 2:** Often the number of heads is chosen as a power of 2 due to potential computational optimizations.

Factors to Consider:

- **Model Size (d_{model}):** Larger models with higher dimensionality generally benefit from more attention heads, as they have the capacity to learn more diverse representations.
- **Task Complexity:** Tasks with intricate relationships or long-range dependencies might benefit from a higher number of heads, allowing the model to discern more nuanced patterns.
- **Dataset Size:** With smaller datasets, using too many heads could lead to overfitting. It might be better to start with a smaller number and increase it if necessary.
- **Computational Constraints:** More attention heads mean more computations and memory requirements. Be mindful of the available hardware and consider a trade-off between representational power and computational cost.

It's All About Experimentation

Unfortunately, there's no single optimal number that suits all scenarios. The best approach is to treat the number of heads as a hyperparameter and experiment. Here's how you can go about it:

- **Start with a reasonable baseline** (e.g., 8 heads for a model with $d_{\text{model}} = 512$).
- **Experiment with variations:** Try a smaller and a larger number of heads, observing how it impacts model performance.
- **Monitor for overfitting:** If using many heads leads to improved training performance but significantly degrades validation performance, it might be a sign of overfitting.

Key Point: The optimal number of heads in multi-head attention is a balancing act between capturing diverse relationships and computational efficiency, often determined through experimentation with your specific task and dataset.

Learning Position with Positional Encoding

Consider the input sentence *'I am good'*. In RNNs, we feed the sentence to the network word by word. That is, first the word *'I'* is passed as input, next the word *'am'* is passed, and so on. We feed the sentence word by word so that our network understands the sentence completely. But with the transformer

network, we don't follow the recurrence mechanism. So, instead of feeding the sentence word by word, we feed all the words in the sentence parallel to the network. Feeding the words in parallel helps in decreasing the training time and also helps in learning the long-term dependency.

Why is word order important?

However, the problem is since we feed the words parallel to the transformer, how will it understand the meaning of the sentence if the word order is not retained? To understand the sentence, the word order (position of the words in the sentence) is important, right? Yes, the word order is very important as it helps to understand the position of each word in a sentence, which in turn helps to understand the meaning of the sentence.

So, we should give some information about the word order to the transformer so that it can understand the sentence. How can we do that? Let's explore this in more detail now.

Sending word order to the transformer

For our given sentence, '*I am good*', first, we get the embeddings for each word in our sentence. Let's represent the embedding dimension as d_{model} . Say the embedding dimension, d_{model} is 4. Then, our input matrix dimension will be:

$$[\text{sentence length} \times \text{embedding dimension}] = [3 \times 4]$$

We represent our input sentence '*I am good*' using the input matrix X (embedding matrix). Let the input matrix X be the following:

	I	1.769	2.22	3.4	5.8	x_1
$X =$	am	7.3	9.9	8.5	7.1	x_2
	good	9.1	7.1	0.85	10.1	x_3
Input matrix (embedding matrix)						

Input matrix

Now, if we pass the preceding input matrix X directly to the transformer, it cannot understand the word order. So, instead of feeding the input matrix directly to the transformer, we need to add some information indicating the word order (position of the word) so that our network can understand the meaning of the sentence. To do this, we introduce a technique called positional encoding.

Positional encoding

Positional encoding, as the name suggests, is an encoding indicating the position of the word in a sentence (word order).

The dimension of the positional encoding matrix, P , is the same dimension as the input matrix X . Now, before feeding the input matrix (embedding matrix) to the transformer directly, we include the positional encoding. So, we simply add the positional encoding matrix P to the embedding matrix X and then feed it as input to the network. So, now our input matrix will have not only the embedding of the word but also the position of the word in the sentence:

$$\begin{array}{c}
 \mathbf{X} = \begin{bmatrix} 1.769 & 2.22 & 3.4 & 5.8 \\ 7.3 & 9.9 & 8.5 & 7.1 \\ 9.1 & 7.1 & 0.85 & 10.1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.841 & 0.54 & 0.01 & 0.99 \\ 0.909 & -0.416 & 0.02 & 0.99 \end{bmatrix} \\
 \mathbf{X} \qquad \qquad \mathbf{P} \\
 = \begin{bmatrix} 1.769 & 3.22 & 3.4 & 6.8 \\ 8.14 & 10.44 & 8.51 & 8.09 \\ 10.0 & 6.68 & 0.87 & 11.09 \end{bmatrix}
 \end{array}$$

Adding an input matrix and positional encoding matrix

Computing positional encoding matrix

Now, the ultimate question is how exactly is the positional encoding matrix computed? The authors of the transformer paper "[Attention Is All You Need]" have used the sinusoidal function for computing the positional encoding, as shown:

$$P(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$P(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

In the preceding equation, *pos* implies the position of the word in a sentence, and *i* implies the position of the embedding. Let's understand the preceding equations with an example. By using the preceding equations, we can write the following:

$$\mathbf{P} = \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} \sin\left(\frac{pos}{10000^0}\right) & \cos\left(\frac{pos}{10000^0}\right) & \sin\left(\frac{pos}{10000^{3/4}}\right) & \cos\left(\frac{pos}{10000^{3/4}}\right) \\ \sin\left(\frac{pos}{10000^0}\right) & \cos\left(\frac{pos}{10000^0}\right) & \sin\left(\frac{pos}{10000^{3/4}}\right) & \cos\left(\frac{pos}{10000^{3/4}}\right) \\ \sin\left(\frac{pos}{10000^0}\right) & \cos\left(\frac{pos}{10000^0}\right) & \sin\left(\frac{pos}{10000^{3/4}}\right) & \cos\left(\frac{pos}{10000^{3/4}}\right) \end{bmatrix}$$

Computing the positional encoding matrix

As we can observe from the preceding matrix, in the positional encoding, we use the sin function when *i* is even and the cos function when *i* is odd. Simplifying the preceding matrix, we can write the following:

$$\mathbf{P} = \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} \sin(pos) & \cos(pos) & \sin\left(\frac{pos}{100}\right) & \cos\left(\frac{pos}{100}\right) \\ \sin(pos) & \cos(pos) & \sin\left(\frac{pos}{100}\right) & \cos\left(\frac{pos}{100}\right) \\ \sin(pos) & \cos(pos) & \sin\left(\frac{pos}{100}\right) & \cos\left(\frac{pos}{100}\right) \end{bmatrix}$$

Computing the positional encoding matrix

We know that in our input sentence, the word 'I' is at the 0th position, 'am' is at the 1st position, and 'good' is at the 2nd position. Substituting the *pos* value, we can write the following:

	I	$\sin(0)$	$\cos(0)$	$\sin(0/100)$	$\cos(0/100)$
P =	am	$\sin(1)$	$\cos(1)$	$\sin(1/100)$	$\cos(1/100)$
	good	$\sin(2)$	$\cos(2)$	$\sin(2/100)$	$\cos(2/100)$

Computing the positional encoding matrix

Thus, our final positional encoding matrix, P , is given as follows:

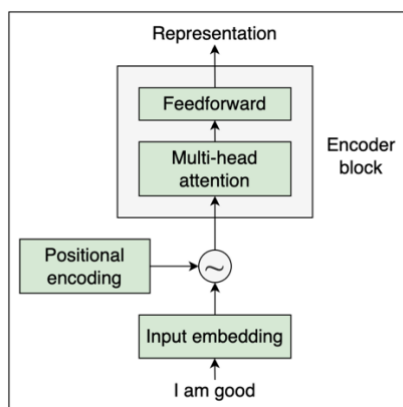
	I	0	1	0	1
P =	am	0.841	0.540	0.009	0.999
	good	0.909	-0.416	0.019	0.999

Positional encoding matrix

After computing the positional encoding P we simply perform element-wise addition with the embedding matrix X and feed the modified input matrix to the encoder.

Encoder architecture for positional encoding

Now, let's revisit our encoder architecture. A single encoder block is shown in the following figure. As we can observe, before feeding the input directly to the encoder, we first get the input embedding (embedding matrix), and then we add the positional encoding to it, and then we feed it as input to the encoder:

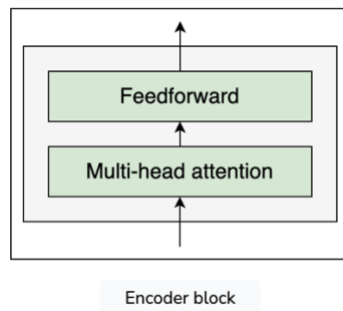


A single encoder block

Encoder: Feedforward and Add and Norm Component

Feedforward network

The feedforward network sublayer in an encoder block is shown in the following figure:

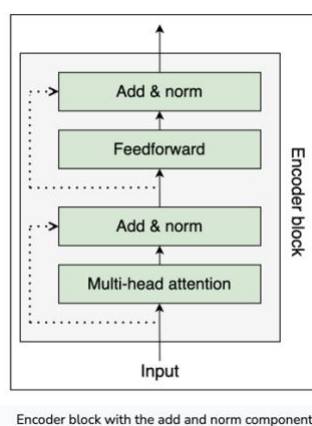


The feedforward network consists of two dense layers with ReLU activations. The parameters of the feedforward network are the same over the different positions of the sentence and different over the encoder blocks.

Add and norm component

One more important component in our encoder is the add and norm component. It connects the input and output of a sublayer. That is, as shown in the following figure (dotted lines), we can observe that the add and norm component does the following:

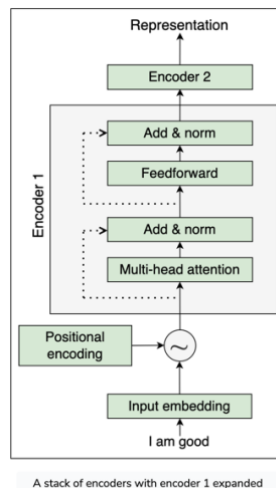
- Connects the input of the multi-head attention sublayer to its output.
- Connects the input of the feedforward sublayer to its output.



The add and norm component is basically a residual connection followed by layer normalization. **Layer normalization** promotes faster training by preventing the values in each layer from changing heavily.

Putting all the Encoder Components Together

The following figure shows the stack of two encoders; only encoder 1 is expanded to reduce the clutter:



Working of the encoder

From the preceding figure, we can understand the following:

1. First, we convert our input to an input embedding (embedding matrix), and then add the position encoding to it and feed it as input to the bottom-most encoder (encoder 1).
2. Encoder 1 takes the input and sends it to the multi-head attention sublayer, which returns the attention matrix as output.
3. We take the attention matrix and feed it as input to the next sublayer, which is the feedforward network. The feedforward network takes the attention matrix as input and returns the encoder representation as output.
4. We take the output obtained from encoder 1 and feed it as input to the encoder above it (encoder 2).
5. Encoder 2 carries the same process and returns the encoder representation of the given input sentence as output.

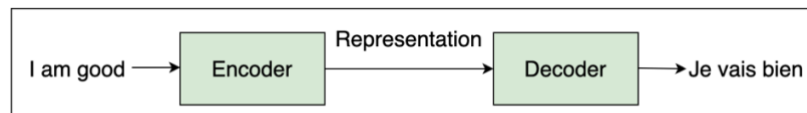
Multiple encoders in the transformer

We can stack N number of encoders one above the other. The output (encoder representation) obtained from the final encoder (topmost encoder) will be the representation of the given input sentence. Let's denote the encoder representation obtained from the final encoder (in our example, it is encoder 2) as R .

We take this encoder representation R , obtained from the final encoder (encoder 2) and feed it as input to the decoder. The decoder takes the encoder representation R as input and tries to generate the target sentence.

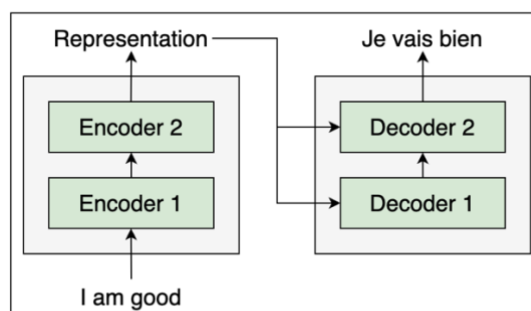
Understanding the Decoder of the Transformer

Suppose we want to translate the English sentence (source sentence) 'I am good' to the French sentence (target sentence) 'Je vais bien'. To perform this translation, we feed the source sentence 'I am good' to the encoder. The encoder learns the representation of the source sentence. We've learned how exactly the encoder learns the representation of the source sentence. Now, we take this encoder's representation and feed it to the decoder. The decoder takes the encoder representation as input and generates the target sentence 'Je vais bien', as shown in the following figure:



Encoder and decoder of the transformer

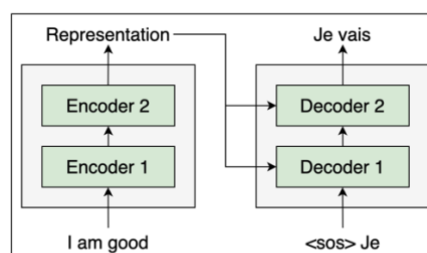
We learned earlier that instead of having one encoder, we can have a stack of N encoders. Similar to the encoder, we can also have a stack of N decoders. For simplicity, let's set $N=2$. As shown in the following figure, the output of one decoder is sent as the input to the decoder above it. We can also observe that the encoder's representation of the input sentence (encoder's output) is sent to all the decoders. Thus, a decoder receives two inputs: one is from the previous decoder, and the other is the encoder's representation (encoder's output):



A stack of encoders and decoders

How the decoder generates the target sentence

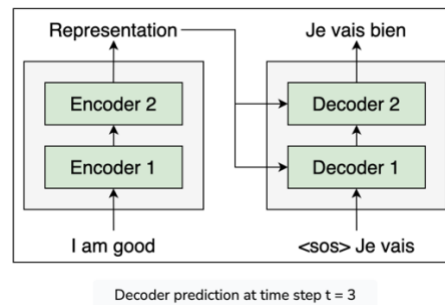
Okay, but how exactly does the decoder generate the target sentence? Let's explore that in more detail. At time step $t=1$, the input to the decoder will be $\langle \text{sos} \rangle$, which indicates the start of the sentence. The decoder takes $\langle \text{sos} \rangle$ as input and generates the first word in the target sentence, which is 'Je', as shown in the following figure:



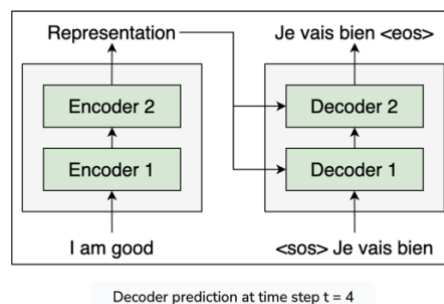
Decoder prediction at time step $t = 2$

At time step $t=2$, along with the current input, the decoder takes the newly generated word from the previous time step, $t-1$, and tries to generate the next word in the sentence. Thus, the decoder takes $\langle \text{sos} \rangle$ and 'Je' (from the previous step) as input and tries to generate the next word in the target sentence, as shown in the following figure:

At time step $t=3$, along with the current input, the decoder takes the newly generated word from the previous time step, $t-1$, and tries to generate the next word in the sentence. Thus, the decoder takes $\langle \text{sos} \rangle$, 'Je', and 'vais' (from the previous step) as input and tries to generate the next word in the sentence, as shown in the following figure:



Similarly, the decoder combines the newly generated word with the input on every time step and predicts the next word. Thus, at time step $t=4$, the decoder takes <sos> , 'Je', 'vais', and 'bien' as input and tries to generate the next word in the sentence, as shown in the following figure:

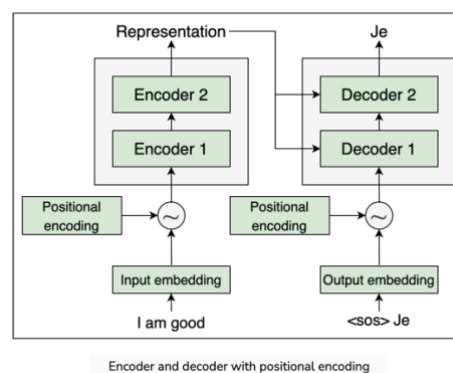


As we can observe from the preceding figure, once the <eos> token, which indicates the end of sentence, is generated, it implies that the decoder has completed generating the target sentence.

Decoder architecture with positional encoding

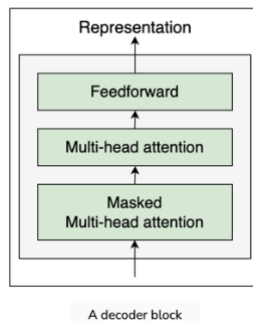
In the encoder section, we learned that we convert the input into an embedding matrix and add the positional encoding to it, and then feed it as input to the encoder. Similarly, here, instead of feeding the input directly to the decoder, we convert it into an embedding, add the positional encoding to it, and then feed it to the decoder.

For example, as shown in the following figure, say at time step $t=2$, we convert the input into an embedding (we call it an output embedding because here we are computing the embedding of the words generated by the decoder in previous time steps), add the positional encoding to it, and then send it to the decoder:



Components of the decoder

Okay, but the ultimate question is how exactly does the decoder work? What's going on inside the decoder? Let's explore this in detail. A single decoder block with all of its components is shown in the following figure:



From the preceding figure, we can observe that the decoder block is similar to the encoder and here we have three sublayers:

- Masked multi-head attention
- Multi-head attention
- Feedforward network

Similar to the encoder block, here we have multi-head attention and feedforward network sublayers. However, here we have two multi-head attention sublayers and one of them is masked.

Masked Multi-Head Attention

In our English-to-French translation task, say our training dataset looks like the one shown here:

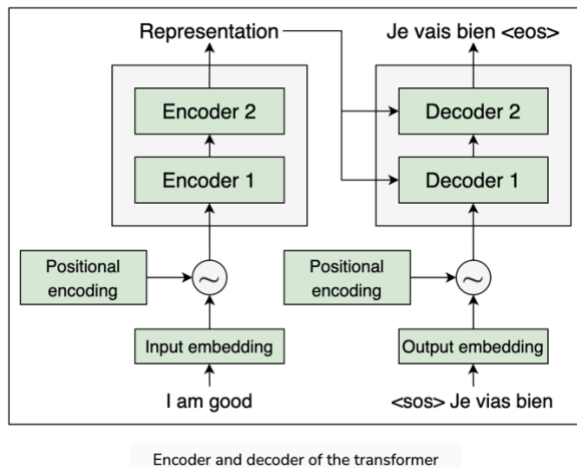
A sample training set

Source sentence	Target sentence
I am good	Je vais bien
Good morning	Bonjour
Thank you very much	Merci beaucoup

By looking at the preceding dataset, we can understand that we have source and target sentences. We saw how the decoder predicts the target sentence word by word in each time step and that happens only during testing.

During training, since we have the right target sentence, we can just feed the whole target sentence as input to the decoder but with a small modification. We learned that the decoder takes the input `< sos >` as the first token, and combines the next predicted word to the input on every time step for predicting the target sentence until the `< eos >` token is reached. So, we can just add the `< sos >` token to the beginning of our target sentence and send that as an input to the decoder.

Say we are converting the English sentence *'I am good'* to the French sentence *'Je vais bien'*. We can just add the `< sos >` token to the beginning of the target sentence and send `< sos > Je vais bien` as an input to the decoder, and then the decoder predicts the output as `Je vais bien < eos >`, as shown in the following figure:



But how does this work? Isn't this kind of ambiguous? Why do we need to feed the entire target sentence and let the decoder predict the shifted target sentence as output? Let's explore this in more detail.

We learned that instead of feeding the input directly to the decoder, we convert it into an embedding (output embedding matrix) and add positional encoding, and then feed it to the decoder. Let's suppose the following matrix, X , is obtained as a result of adding the output embedding matrix and positional encoding:

	<sos>	7.9	3.5	...	16.1	x_1
$X =$	Je	8.1	4.4	...	83.1	x_2
	vais	17	0.54	...	6.12	x_3
	bien	11.12	11.12..		22.1	x_4

Input matrix

Now, we feed the preceding matrix X , to the decoder. The first layer in the decoder is the masked multi-head attention. This works similarly to the multi-head attention mechanism we learned about with the encoder, but with a small difference.

Computing query, key, and value matrices

To perform self-attention, we create three new matrices, called query Q , key K , and value V . Since we are computing multi-head attention, we create hh number of query, key, and value matrices. Thus, for a head i , the query Q_i , key K_i , and value V_i , matrices can be created by multiplying X by the weight matrices W_iQ , W_iK , W_iV , respectively.

How masked multi-head attention works

Now, let's see how masked multi-head attention works. Our input sentence to the decoder is <sos>Je vais bien . We learned that the self-attention mechanism relates a word to all the words in the sentence to understand more about each word. But there is a small catch here. During test time, the decoder will only have the words generated until the previous step as input. For example, during testing, say at time step $t=2$, the decoder will only have the input words as $[\text{<sos>}, \text{Je}]$ and it will not have any other words. So, we have to train our model in the same fashion. Thus, our attention mechanism should relate the words only until the word 'Je' and not the other words. To do this, we can mask all the words on the right that are not predicted by our model yet.

Say we want to predict the word next to the word `<sos>`. In this case, the model should see only the words up to `<sos>`, so we mask all the words on the right of `<sos>`. Say we want to predict the word next to the word `'Je'`. In this case, the model should see only the words up to `'Je'`, so we mask all the words to the right of `'Je'`, and the same applies for other rows, as shown in the following figure:

<code><sos></code>	mask	mask	mask
<code><sos></code>	Je	mask	mask
<code><sos></code>	Je	vais	mask
<code><sos></code>	Je	vais	bien

Masking the values

Masking words like this help the self-attention mechanism to attend only to the words that would be available to the model during testing. Okay, but how exactly we can perform this masking? We know that for a head, i , the attention matrix, Z_i , is computed as follows:

$$Z_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i$$

Step 1: Calculating the similarity score

The first step in computing the attention matrix is computing the dot product between the query and key matrices. The following shows the result of the dot product between the query and the key matrix. Note that the values used here are arbitrary, used here just to get a good understanding:

		<code><sos></code>	Je	vais	bien
	<code><sos></code>	73	60	10	45
$Q_i K_i^T =$	Je	40	99	25	70
	vais	58	40	83	10
	bien	12	11	15	80

Dot product of the query and key matrices

Step 2: Obtaining stable gradient

The next is to divide the $Q_i K_i^T$ matrix by the dimension of key vector d_k . Suppose the following is the result of $d_k Q_i K_i^T$:

		<code><sos></code>	Je	vais	bien
	<code><sos></code>	9.125	7.5	1.25	5.625
$\frac{Q_i K_i^T}{\sqrt{d_k}} =$	Je	5.0	12.37	3.12	8.75
	vais	7.25	5.0	10.37	1.25
	bien	1.5	1.37	1.87	10.0

Dividing $Q_i K_i^T$ by the square root of d_k

Step 3: Masking and normalizing the gradients

Next, we apply the softmax function to the preceding matrix and normalize the scores. But before applying the softmax function, we need to mask the values. For example, look at the first row of our matrix. To predict the word next to the word $\langle \text{sos} \rangle$, our model should not attend all the words to the right of $\langle \text{sos} \rangle$ (as this will not be available during test time). So, we can mask all the words to the right of $\langle \text{sos} \rangle$ with $-\infty$:

	$\langle \text{sos} \rangle$	Je	vais	bien
$\langle \text{sos} \rangle$	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	3.12	8.75
vais	7.25	5.0	10.37	1.25
bien	1.5	1.37	1.87	10.0

Masking all the words to the right of $\langle \text{sos} \rangle$ with $-\infty$

Now, let's take a look at the second row of the matrix. To predict the word next to the word Je , our model should not attend all the words to the right of Je (as this will not be available during test time). So, we can mask all the words to the right of Je with $-\infty$:

	$\langle \text{sos} \rangle$	Je	vais	bien
$\langle \text{sos} \rangle$	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	$-\infty$	$-\infty$
vais	7.25	5.0	10.37	1.25
bien	1.5	1.37	1.87	10.0

Masking all the words to the right of Je with $-\infty$

Similarly, we can mask all the words to the right of $vais$ with $-\infty$ as shown:

	$\langle \text{sos} \rangle$	Je	vais	bien
$\langle \text{sos} \rangle$	9.125	$-\infty$	$-\infty$	$-\infty$
Je	5.0	12.37	$-\infty$	$-\infty$
vais	7.25	5.0	10.37	$-\infty$
bien	1.5	1.37	1.87	10.0

Masking all the words in the right of $vais$ with $-\infty$

Now, we can apply the softmax function to the preceding matrix.

Step 4: Computing the attention matrix

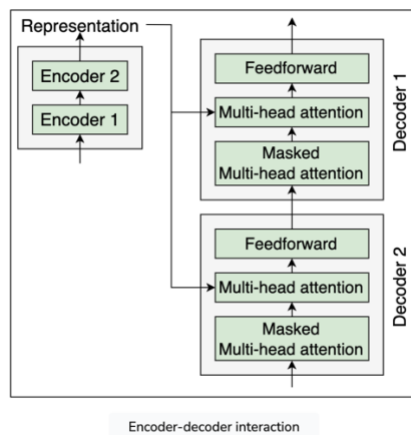
After applying the softmax function to the preceding matrix, we multiply the result by the value matrix, V_i , and obtain the final attention matrix Z_i . Similarly, we can compute h number of attention matrices, concatenate them, and multiply the result by a new weight matrix, W_O , and create the final attention matrix, M , as shown:

$$M = \text{Concatenate}(Z_1, Z_2, \dots, Z_i, \dots, Z_h) W_O$$

Now, we feed this final attention matrix, M , to the next sublayer in our decoder, which is another multi-head attention layer.

Multi-Head Attention

The following figure shows the transformer model with both the encoder and decoder. As we can observe, the multi-head attention sublayer in each decoder receives two inputs: one is from the previous sublayer, masked multi-head attention, and the other is the encoder representation:



Let's represent the encoder representation with R and the matrix obtained as a result of the masked multi-head attention sublayer with M . Since here we have an interaction between the encoder and decoder, this layer is also called an **encoder-decoder attention layer**.

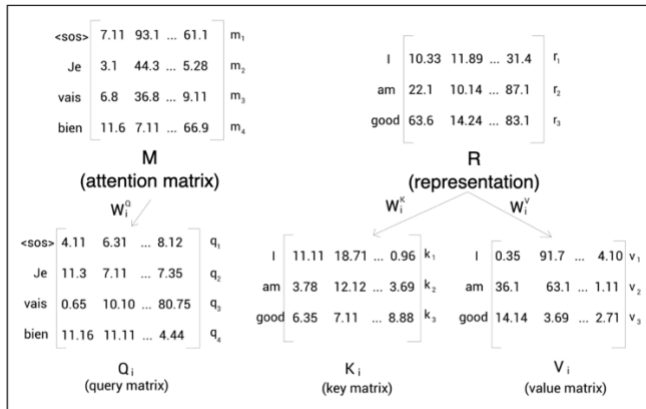
How the multi-head attention layer works

Now, let's look into the details and learn how exactly this multi-head attention layer works. The first step in the multi-head attention mechanism is creating the query, key, and value matrices. We learned that we can create the query, key, and value matrices by multiplying the input matrix by the weight matrices. But in this layer, we have two input matrices: one is R (the encoder representation) and the other is M (the attention matrix from the previous sublayer). So, which one should we use?

Computing query, key, and value matrices

We create the query matrix, Q , using the attention matrix, M , obtained from the previous sublayer and we create the key and value matrices using the encoder representation R . Since we are performing the multi-head attention mechanism, for head i , we do the following:

- The query matrix, Q_i , is created by multiplying the attention matrix, M , by the weight matrix, W_iQ .
- The key and value matrices are created by multiplying the encoder representation R , by the weight matrices, W_iK and W_iV , respectively. This is shown in the following figure:

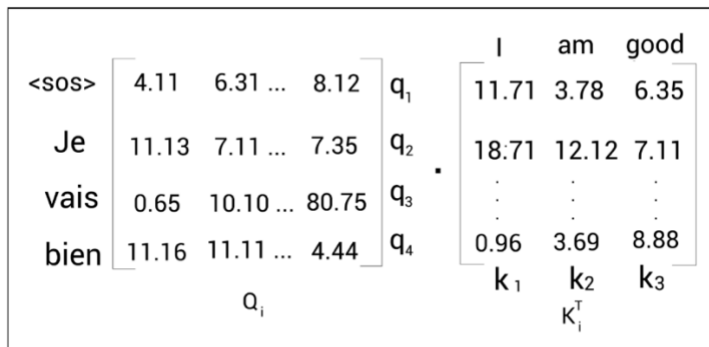


Creating query, key, and value matrices

But why do we have to do this? Why do we obtain the query matrix from M and key and value matrices from R ? The query matrix essentially holds the representation of our target sentence since it is obtained from M and the key and value matrices hold the representation of the source sentence since it is obtained from R . But how and why exactly is this useful? Let's understand this by calculating the self-attention step by step.

Step 1: Calculating the similarity score

The first step in self-attention is to compute the dot product between the query and key matrices. The query and key matrices are shown in the following figure. As we can observe, since the query matrix is obtained from M , it holds the representation of the target sentence, and since the key matrix is obtained from R , it holds the representation of the input sentence. Note that the values used here are arbitrary, used here just to get a good understanding:



Query and key matrices

The following shows the result of the dot product between the query and key matrices:

$$\begin{array}{c}
\begin{matrix} & \text{I} & \text{am} & \text{good} \\ \text{<sos>} & 4.11 & 6.31 & \dots & 8.12 & Q_1 \\ \text{Je} & 11.13 & 7.11 & \dots & 7.35 & Q_2 \\ \text{vais} & 0.65 & 10.10 & \dots & 80.75 & Q_3 \\ \text{bien} & 11.16 & 11.11 & \dots & 4.44 & Q_4 \end{matrix} \cdot \begin{matrix} & \text{I} & \text{am} & \text{good} \\ 11.71 & 3.78 & 6.35 \\ 16.71 & 12.12 & 7.11 \\ \vdots & \vdots & \vdots \\ 0.96 & 3.69 & 8.88 \end{matrix} \\
Q_i \cdot K_i^T = \begin{matrix} & \text{I} & \text{am} & \text{good} \\ \text{<sos>} & q_1 k_1 & q_1 k_2 & q_1 k_3 \\ \text{Je} & q_2 k_1 & q_2 k_2 & q_2 k_3 \\ \text{vais} & q_3 k_1 & q_3 k_2 & q_3 k_3 \\ \text{bien} & q_4 k_1 & q_4 k_2 & q_4 k_3 \end{matrix} \\
Q_i K_i^T = \begin{matrix} & \text{I} & \text{am} & \text{good} \\ \text{<sos>} & 91 & 60 & 77 \\ \text{Je} & 96 & 63 & 12 \\ \text{vais} & 41 & 111 & 48 \\ \text{bien} & 36 & 45 & 65 \end{matrix}
\end{array}$$

Dot product between the query and key matrices

By looking at the preceding matrix, $Q_i K_i^T$, we can understand the following:

- From the first row of the matrix, we can observe that we are computing the dot product between the query vector $q_1(\text{<sos>})$ and all the key vectors— $k_1(\text{I})$, $k_2(\text{am})$, and $k_3(\text{good})$. Thus, the first row indicates how similar the target word <sos> is to all the words in the source sentence ('I', 'am', and 'good').
- Similarly, from the second row of the matrix, we can observe that we are computing the dot product between the query vector $q_2(\text{Je})$ and all the key vectors— $k_1(\text{I})$, $k_2(\text{am})$, and $k_3(\text{good})$. Thus, the second row indicates how similar the target word 'Je' is to all the words in the source sentence ('I', 'am', and 'good').
- The same applies to all other rows. Thus, computing $Q_i K_i^T$ helps us to understand how similar our query matrix (target sentence representation) is to the key matrix (source sentence representation).

Step 2: Obtaining the stable gradients

The next step in the multi-head attention matrix is to divide $Q_i K_i^T$ by dk .

Step 3: Normalizing the gradients

Then, we apply the softmax function and obtain the score matrix, $\text{softmax}(dk Q_i K_i^T) V_i$.

Step 4: Computing the attention matrix

Next, we multiply the score matrix by the value matrix, V_i , that is, $\text{softmax}(dk Q_i K_i^T) V_i$ obtain the attention matrix, Z_i , as shown:

$$Z_i = \begin{matrix} & \text{I} & \text{am} & \text{good} \\ \text{<sos>} & 0.84 & 0.017 & 0.14 \\ \text{Je} & 0.98 & 0.02 & 0.0 \\ \text{vais} & 0.0 & 1.0 & 0.0 \\ \text{bien} & 0.0 & 0.0 & 1.0 \end{matrix} \cdot \begin{matrix} \text{I} & 0.35 & 91.7 & \dots & 4.10 \\ \text{am} & 36.1 & 63.1 & \dots & 1.11 \\ \text{good} & 14.14 & 3.69 & \dots & 2.71 \end{matrix} \\
\text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

Computing the attention matrix

Say we have the following:

$$Z_1 = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix} = \begin{bmatrix} \text{<sos>} \\ \text{Je} \\ \text{vais} \\ \text{bien} \end{bmatrix}$$

Result of attention matrix

The attention matrix, Z_i , of the target sentence is computed by taking the sum of value vectors weighted by the scores. To get some clarity, let's see how the self-attention value of the word 'Je', z_2 , is computed:

$$Z_2 = 0.98 \begin{bmatrix} 0.35 & 91.7 & \dots \end{bmatrix} + 0.02 \begin{bmatrix} 36.1 & 63.1 & \dots \end{bmatrix} + 0.0 \begin{bmatrix} 14.14 & 3.69 & \dots \end{bmatrix}$$

$V_1(I) \qquad \qquad \qquad V_2(am) \qquad \qquad \qquad V_3(good)$

Self-attention of the word Je

As shown, z_2 , the self-attention of the word 'Je' is computed as the sum of the value vectors weighted by the scores. Thus, the value of z_2 will contain 98% of the values from the value vector $v_1(I)$ and 2% of the values from the value vector $v_2(am)$. This basically helps the model to understand that the target word 'Je' means the source word 'I'.

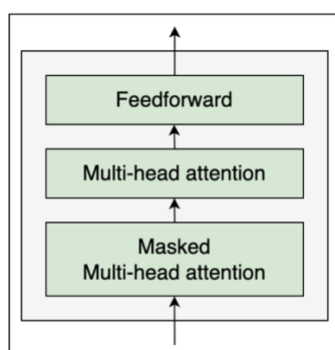
Similarly, we can compute h number of attention matrices, concatenate them, and multiply the result by a new weight matrix, W_0 , and create the final attention matrix, as shown:

$$\text{Multi-head attention} = \text{Concatenate}(Z_1, Z_2, \dots, Z_i, \dots, Z_h) W_0$$

Now, we feed this final attention matrix to the next sublayer in our decoder, which is a feedforward network.

Feedforward network

The next sublayer in the decoder is the feedforward network, as shown in the following figure:

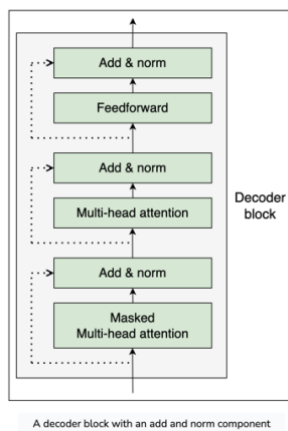


A decoder block

The feedforward layer in the decoder works exactly the same as what we learned in the encoder. Next, we will look into the add and norm component.

Add and norm component

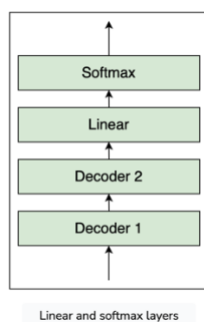
Just like we learned with the encoder, the add and norm component connects the input and output of a sublayer, as shown in the following figure:



Next, we will look at the linear and softmax layers.

Linear and softmax layers

Once the decoder learns the representation of the target sentence, we feed the output obtained from the topmost decoder to the linear and softmax layers, as shown in the following figure:



The linear layer generates the logits whose size is equal to our vocabulary size. Suppose our vocabulary consists of only the following three words:

Vocabulary = [bien, Je, vais]

Now, the logits returned by the linear layer will be a vector of size 3. Next, we convert the logits into a probability using the softmax function, and then the decoder outputs the word whose index has a high probability value. Let's understand this with an example.

Suppose the input to the decoder are the words <sas> and 'Je'. Now, the decoder needs to predict the next word in our target sentence. So, we take the output of the topmost decoder and feed it to the linear layer. The linear layer generates the logits vector whose size is our vocabulary size. Let the logits returned by the linear layer be the following:

logits=[45,40,49]

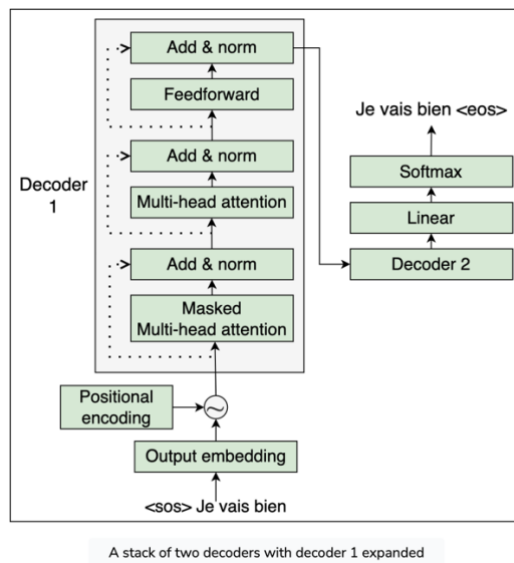
Now, we apply the softmax function to the logits generated by the linear layer and obtain the probability:

prob=[0.0179,0.000,0.981]

From the preceding matrix, we can understand that the probability is high at index 2. So, we look for the word that is at index 2 in our vocabulary. Since the word 'vais' is at index 2, our decoder will predict the next word in our target sentence as 'vais'. In this way, the decoder predicts the next word in the target sentence.

Putting All the Decoder Components Together

The following figure shows the stack of two decoders; only decoder 1 is expanded to reduce the clutter:



How the decoder works

From the preceding figure, we can understand the following:

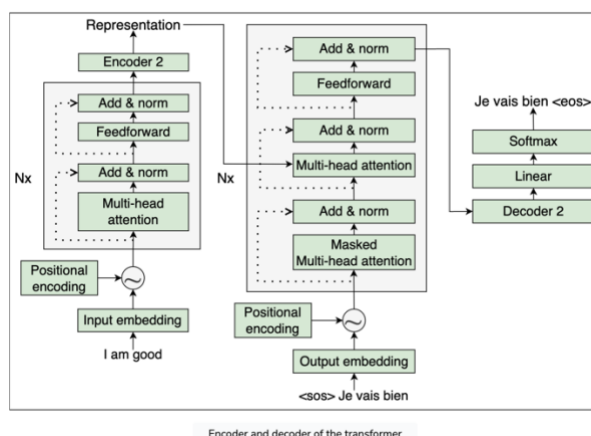
1. We convert the input to the decoder into an embedding matrix and then add the position encoding to it and feed it as input to the bottom-most decoder (decoder 1).
2. The decoder takes the input and sends it to the masked multi-head attention layer, which returns the attention matrix, M , as output.
3. We take the attention matrix, M , and also the encoder representation, R , and feed them as input to the multi-head attention layer (encoder-decoder attention layer), which again outputs the new attention matrix.
4. We take the attention matrix obtained from the encoder-decoder attention layer and feed it as input to the next sublayer, which is the feedforward network. The feedforward network takes the attention matrix as input and returns the decoder representation as output.
5. We take the output obtained from decoder 1 and feed it as input to the decoder above it (decoder 2).
6. Decoder 2 carries the same process and returns the decoder representation of the target sentence as output.

Multiple decoders in the transformer

We can stack N number of decoders one above the other; the output (decoder representation) obtained from the final decoder will be the representation of the target sentence. Next, we feed the decoder representation of the target sentence to the linear and softmax layers and get the predicted word.

Putting the Encoder and Decoder Together

We feed the decoder representation of the target sentence to the linear and softmax layers and get the predicted word. To give more clarity, the complete transformer architecture with the encoder and decoder is shown in the following figure:



In the preceding figure, N_x denotes that we can stack N number of encoders and decoders. As we can observe, once we feed the input sentence (source sentence), the encoder learns the representation and sends the representation to the decoder, which in turn generates the output sentence (target sentence).

Training the transformer

We can train the transformer network by minimizing the loss function. Okay, but what loss function should we use? We learned that the decoder predicts the probability distribution over the vocabulary and we select the word that has the highest probability as output. So, we have to minimize the difference between the predicted and actual probability distributions. First, how can we find the difference between the two distributions? We can use cross-entropy for that. Thus, we can define our loss function as a cross-entropy loss and try to minimize the difference between the predicted and actual probability distribution. We train the network by minimizing the loss function, and we use Adam as an optimizer.

To prevent overfitting, we apply dropout to the output of each sublayer, the sum of the embeddings, and the positional encoding.

Summary: A Primer on Transformers

Summarized below are the main highlights of what we have learned in this chapter.

- We learned what the transformer model is and how it uses encoder-decoder architecture. We looked into the encoder section of the transformer and learned about different sublayers used in encoders, such as multi-head attention and feedforward networks.
- We learned that the self-attention mechanism relates a word to all the words in the sentence to better understand the word. To compute self-attention, we used three different matrices called the query, key, and value matrices. Following this, we learned how to compute positional

encoding and how it is used to capture the word order in a sentence. Next, we learned how the feedforward network works in the encoder, and then we explored the add and norm component.

- We learned how the decoder works. We explored three sublayers used in the decoder in detail, which are masked multi-head attention, encoder-decoder attention, and feedforward network.
- We learned how the transformer works with the encoder and decoder and how we train the network.